Diffusion Models: From Adding Noise to Generating Images

From Pixel-Space Denoising Diffusion to High-Resolution Latent Diffusion

Thanh Huu Nguyen
Student ID: 24125030

Manh Cuong Do
Student ID: 24125054

Thi Thanh Huyen Phan
Student ID: 24125031

Faculty of Information Technology, University of Science, VNU-HCM, Ho Chi Minh City, Vietnam

Abstract

Diffusion models generate images by learning how to remove noise from data. This report studies how this process works and why performing diffusion in a compressed latent space can reduce its computational cost. We review Denoising Diffusion Probabilistic Models (DDPM) and Latent Diffusion Models (LDM), explain their main equations, and provide a step-by-step numerical example. We also examine small class-conditioned implementations on MNIST, using saved experiment records and generated images. In this setting, latent diffusion requires about 14.19 times fewer counted convolution and linear operations per denoising step. However, its recorded sampling time is 1.715 seconds, compared with 1.592 seconds for pixel diffusion, and it requires an additional autoencoder stage. These results illustrate that lower arithmetic cost does not necessarily lead to lower latency at small resolutions. The examples demonstrate basic generation and class conditioning, but do not establish distributional quality or general text-prompt control. We discuss these limits alongside the effects of compression, training data, copyright, and bias.

Index Terms—Generative AI, Denoising Diffusion Probabilistic Models (DDPM), Latent Diffusion Models (LDM), Variational Autoencoder, Cross-Attention.

1 Introduction

Artificial intelligence (AI) image synthesis is the process of generating new images by learning patterns from existing data. It is used in digital content creation, image editing, and visual design. However, generating realistic, diverse, and high-resolution images remains challenging. Generative Adversarial Networks (GANs) can generate images quickly, but their training may be unstable or produce limited variation. Autoregressive models provide an explicit probabilistic description of images, although sequential generation can be slow [1], [2], [3].

Diffusion models use a different approach. A fixed forward process gradually adds noise to training images, while a neural network learns information needed to reverse this process. Once trained, the model can generate new images starting from random noise. Ho et al. [4] show that predicting added noise with a simplified objective can produce high-quality samples. Their pixel-space Denoising Diffusion Probabilistic Model (DDPM) requires many denoising steps, making generation expensive. Rombach et al. [5] reduce the cost of each step by applying diffusion to compressed image representations in Latent Diffusion Models (LDM).

This report addresses the question: how can learning to remove noise enable a model to generate new images? We explain the forward and reverse processes, calculate a one-dimensional example, and examine five noise levels on a 28×28 image. We then describe latent compression and cross-attention conditioning, and compare small pixel-space and latent-space implementations using saved MNIST experiment records. The comparison considers representation size, estimated arithmetic, recorded runtime, reconstruction, and conditioning. These experiments illustrate the mechanisms rather than reproduce the original papers' image-quality results. We also discuss limitations related to compression, data, copyright, and bias.

2 Background and Paper Review

2.1 Discriminative and Generative Models

Given an image x and label y, a discriminative model predicts $p(y \mid x)$ or a decision boundary, for example to classify a digit. A generative model instead models a data distribution such as $p(x)$ or $p(x \mid y)$ and draws new samples from it. Conditional generation can therefore produce an image consistent with a class label or text description. Classification alone does not provide this sampling capability.

2.2 Qualitative Comparison of Generative Models

GANs train a generator against a discriminator, enabling fast generation but introducing a difficult optimization problem and a risk of mode collapse [1], [2]. Autoregressive models factorize a joint distribution into conditional probabilities and generate components sequentially [3]. Variational Autoencoders (VAEs) learn a probabilistic encoder and decoder with a latent prior; generation draws from that prior and decodes a sample [6]. Diffusion models learn denoising across noise levels and use repeated network evaluations for generation [4], [7]. Table 1 summarizes typical trade-offs, not a universal ranking or a controlled benchmark.

Table 1: Qualitative synthesis by the authors from [1], [2], [3], [6], and [4]. Outcomes depend on architecture, data, and evaluation.

Criterion	GAN	Autoregressive	VAE	Diffusion
Learning	Adversarial	Conditional likelihood	Variational bound	Denoising objective
Quality	Can be sharp	Can be high	May be smoother	Can be high
Training	Can be unstable	Direct likelihood objective	Variational optimization	No adversarial game
Coverage	Mode-collapse risk	Depends on fit	Depends on prior and fit	Depends on fit; no guarantee
Sampling	One generator pass	Sequential components	One decoder pass	Many denoising steps

2.3 Review of Core Papers: From DDPM to LDM

Sohl-Dickstein et al. [7] introduced the earlier diffusion probabilistic framework. Ho, Jain, and Abbeel [4] develop an effective noise-prediction parameterization and a simplified training objective, connecting diffusion to denoising score matching and Langevin-like sampling. Their unconditional CIFAR-10 result has Inception Score (IS) 9.46 ± 0.11 and Fréchet Inception Distance (FID) 3.17 (Table 1 of their paper). FID compares feature distributions of real and generated samples; lower values indicate closer agreement under that evaluation. The paper reports competitive 256×256 LSUN samples, but repeated pixel-space denoising remains costly. These published findings are not reproduced here.

Rombach et al. [5] separate perceptual compression from diffusion-based generation. A pretrained autoencoder provides a smaller spatial representation, and the diffusion model learns its distribution. Their cross-attention interface supports token-based conditions such as text and layouts; spatially aligned conditions can also be concatenated. Compression introduces a trade-off: small downsampling factors leave expensive representations, whereas excessive compression loses detail (their Section 4.1 and Figures 5-6). Their Table 6 provides a matched inpainting comparison: on Places at 256×256, LDM-1 and KL-regularized LDM-4 with attention have training throughputs of 0.11 and 0.32 samples/s and sampling throughputs of 0.26 and 0.97 samples/s. The settings have fixed model parameter counts; hardware/batch details are specified in the paper's supplement. These task-specific results support efficiency benefits without defining a universal speedup.

For class-conditioned ImageNet generation, their Table 3 reports FID 10.56 for LDM-4 and 3.60 for LDM-4-G with classifier-free guidance (scale 1.5, 250 sampling steps). These are separate from DDPM's CIFAR-10 result and cannot be compared as a common quality ranking. Together, the papers motivate two distinct design choices: learning a denoising-based generator and choosing an efficient representation in which to run it. Faster samplers such as DDIM can additionally reduce network evaluations in either space [8].

3 Method and Worked Example

3.1 DDPM: Forward Noising and Reverse Denoising

3.1.1 Definitions and Forward Process

We follow Ho et al. [4] (Sections 2-3). A clean image is a vector x_0 with $D = H \times W \times C$ coordinates, where H, W, and C denote height, width, and channels. Integer intensities are scaled by $x_0 = \frac{x_{\text{raw}}}{127.5} - 1$ into $[-1, 1]^D$. This sets a consistent numerical range; it does not guarantee zero empirical mean or unit variance. The states after noising lie in $\mathbb{R}^D$, so noisy coordinates need not remain in that interval. The principal symbols are defined in Table 2.

Table 2: Notation used in the DDPM equations, following [4]. All scaled intensities, noise coordinates, and variances are dimensionless.

Symbol	Meaning and range
q_{data}, x_0	Data distribution and clean sample $x_0 \sim q_{\text{data}}$ in $[-1, 1]^D$.
q, p_θ, θ	Fixed forward distribution, learned generative distribution, and trainable real-valued network parameters.
t, T	Discrete step $t \in \{1, ..., T\}$; $T = 1000$ in our experiments.
β_t, α_t	Forward variance $0 < \beta_t < 1$; $\alpha_t = 1 - \beta_t$.
$\overline{\alpha}_t$	Product $\prod_{s=1}^{t} \alpha_s$, with $\overline{\alpha}_0 = 1$.
$\boldsymbol{I}, \varepsilon$	$D \times D$ identity matrix and independent noise $\varepsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.
$\varepsilon_\theta, \mu_\theta, \sigma_t^2$	Predicted noise and reverse mean in $\mathbb{R}^D$, and fixed reverse variance $\sigma_t^2 \geq 0$.
$\mathbb{E}, \|v\|^2$	Expectation over stated random draws; squared Euclidean norm $\sum_{j=1}^{D} v_j^2$.

The forward process adds independent Gaussian increments. Following Eqs. (2) and (4) of [4],

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \quad q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \boldsymbol{I}\Big). \tag{1}$$

Here $\mathcal{N}(x; \mu, \Sigma)$ denotes a Gaussian density evaluated at x, with mean μ and covariance Σ. The experiments use the linear schedule $\beta_t = 10^{-4} + (t-1)\frac{0.02-10^{-4}}{T-1}$. Thus $\sqrt{\beta_t}$ is the standard deviation of one noise increment. Small steps motivate a Gaussian approximation for learned reverse transitions; they do not make the exact reverse conditional Gaussian for arbitrary data.

The accumulated forward noise has a closed form:

$$q(x_t \mid x_0) = \mathcal{N}\Big(x_t; \sqrt{\overline{\alpha}_t}x_0, (1-\overline{\alpha}_t)\boldsymbol{I}\Big), \tag{2}$$

$$x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon. \tag{3}$$

For this finite schedule, $\overline{\alpha}_{1000} \approx 4.0358 \times 10^{-5}$ and the signal coefficient is approximately 0.00635. The terminal state is therefore close to standard Gaussian noise, with a small residual dependence on the image. The forward distribution is fixed rather than learned. Figure 1 illustrates both directions.

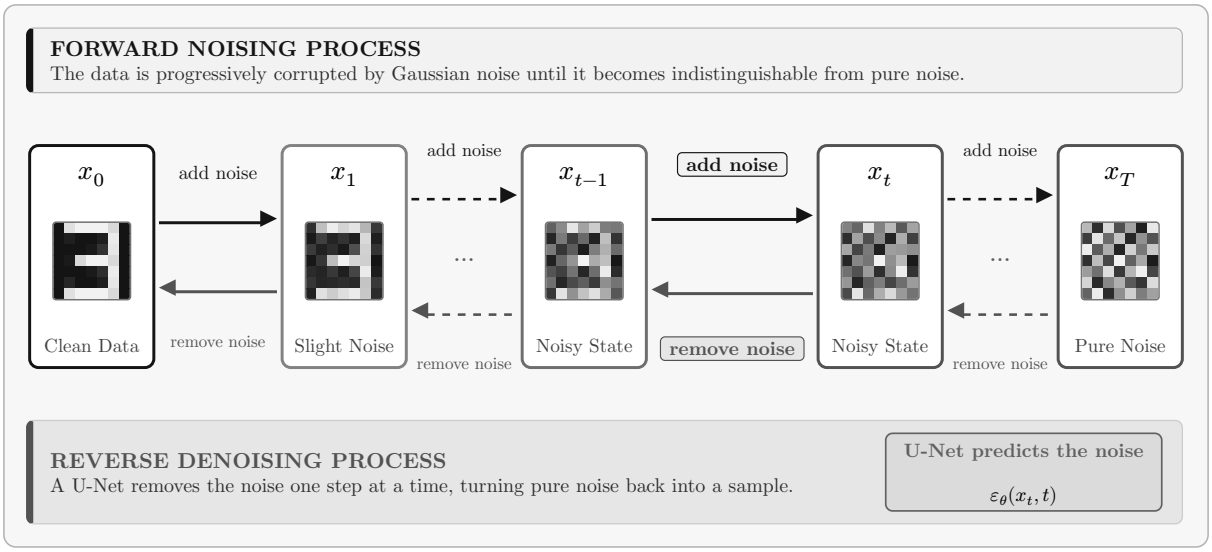


Figure 1: Conceptual forward and reverse chains, drawn by the authors based on [4], Figure 2. The small grids are stylized illustrations, not Gaussian simulations. Reverse sampling generates a new sample rather than recovering a particular original image.

3.1.2 Reverse Model and Noise-Prediction Objective

The learned model starts from $p(x_T) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and uses

$$p_{\theta(x_{0:T})} = p(x_T) \prod_{t=1}^{T} p_{\theta(x_{t-1}|x_t)}, \quad p_{\theta(x_{t-1}|x_t)} = \mathcal{N}\big(x_{t-1}; \mu_{\theta(x_t,t)}, \sigma_t^2 \boldsymbol{I}\big). \tag{4}$$

For $t > 1$, the posterior conditioned on the clean image is Gaussian. Its mean and variance, from Eq. (7) of [4], are

$$\tilde{\mu}_{t(x_t,x_0)} = \frac{\sqrt{\overline{\alpha}_{t-1}}\beta_t}{1-\overline{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}\beta_t. \tag{5}$$

We use $\sigma_t^2 = \tilde{\beta}_t$ during sampling. Noise prediction provides a convenient mean parameterization (their Eq. (11)):

$$\mu_{\theta(x_t,t)} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\overline{\alpha}_t}} \varepsilon_{\theta(x_t,t)} \right). \tag{6}$$

This does not automatically give an unweighted loss. For intermediate steps, the negative evidence lower bound (negative ELBO), which upper-bounds negative log-likelihood, contains terms of the form

$$\mathbb{E}\Big[w_t \big\| \varepsilon - \varepsilon_{\theta(x_t,t)} \big\|^2 \Big] + \text{constant}, \quad w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_{t(1-\overline{\alpha}_t)}}, \quad t > 1. \tag{7}$$

Ho et al. deliberately discard these timestep-dependent weights to obtain their simplified objective (Eqs. (12) and (14) of [4]):

$$L_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\Big[\big\| \varepsilon - \varepsilon_{\theta(x_t,t)} \big\|^2 \Big]. \tag{8}$$

Training samples $x_0 \sim q_{\text{data}}$, t uniformly from $\{1, ..., T\}$, and independent standard Gaussian ε. The network receives x_t from Eq. 3 and t; an optimizer updates θ to reduce the noise error. Our conditional models also receive label y. The code averages squared error over coordinates and the mini-batch; this rescales the sum in Eq. 8 by a constant for each fixed representation size.

Why can this generate new images? Under squared error, the optimal predictor estimates the conditional mean of the injected noise given the noisy input. Across images and noise levels, it learns statistical structure that guides reverse transitions toward likely data. It is not given the original image

during generation and does not need to identify the exact noise realization that produced an arbitrary input. This is the noise-prediction connection to denoising score matching discussed by [4].

3.1.3 Training and Sampling Steps

The following procedures summarize Algorithms 1-2 of [4].

1. Training: draw a mini-batch of clean images and labels, random timesteps, and independent noise.
2. Form each noisy input with Eq. 3; predict its noise using the image, timestep, and label.
3. Average squared prediction errors, backpropagate, and update the network. Repeat over training batches.
4. Sampling: draw $x_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, choose the label if conditional, and iterate $t = T, ..., 1$ using

$$x_{t-1} = \mu_{\theta(x_t,t)} + \sigma_t \xi_t, \quad \xi_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}) \ \text{ for } \ t > 1, \quad \xi_1 = \mathbf{0}. \tag{9}$$

Each reverse-step noise draw is independent. The implementation first estimates $\hat{x}_0 = \frac{x_t - \sqrt{1-\overline{\alpha}_t}\varepsilon_\theta}{\sqrt{\overline{\alpha}_t}}$, clips it to $[-1, 1]$, and uses this estimate in Eq. 5. At $t = 1$ it returns the clean estimate without extra noise. Thus the implemented mean includes clipping, unlike the unclipped algebraic form of Eq. 6. Code index k denotes mathematical step $t = k + 1$.

3.2 Step-by-Step Worked Numerical Example

We use the first MNIST test image labeled 3 [9], with zero-based pixel coordinate $(10, 14)$, following the saved output of `step0_worked_example.py`. Its raw intensity is 236. A CPU PyTorch noise draw with seed 42 supplies the selected coordinate $\varepsilon = -0.714962$. This is one realization, not an expected noise value. The one-dimensional forward step is calculated as follows:

1. Normalize: $x_0 = \frac{236}{127.5} - 1 = 0.850980$.
2. Choose $\beta_1 = 0.0001$, so $\sqrt{1-\beta_1} \approx 0.999950$ and $\sqrt{\beta_1} = 0.010000$.
3. Calculate the conditional mean: $\mu_1 = 0.999950 \times 0.850980 = 0.850938$.
4. Calculate the noise increment: $0.010000 \times (-0.714962) = -0.007150$.
5. Add the terms:

$$x_1 = 0.850938 - 0.007150 = 0.843788. \tag{10}$$

The conditional expectation is $\mathbb{E}[x_1|x_0] = 0.850938$, because $\mathbb{E}[\varepsilon] = 0$; its conditional variance is 0.0001. Repeating the experiment would give different samples around this mean. More generally, for independent noise, $\mathrm{Var}(x_t) = \overline{\alpha}_t \ \mathrm{Var}(x_0) + (1 - \overline{\alpha}_t)$ for each coordinate. It equals one only if the original coordinate has unit variance; normalization alone does not imply that assumption.

At $t = 50$, the recorded $\overline{\alpha}_{50} = 0.971016$ gives

$$x_{50} = 0.985401 \times 0.850980 + 0.170248 \times (-0.714962) = 0.716836. \tag{11}$$

Reusing the same ε couples two valid marginal samples for illustration; it is not a sequential Markov path.

The saved untrained U-Net output at that pixel, $\left[\varepsilon_{\theta(x_1,1,y=3)}\right]_{10,14} = 0.421943$, gives a residual of -1.136905 and squared error 1.292554. Here x_1 in the predictor denotes the whole noisy image. The recorded image-level mean is

$$\ell_{t=1} = \frac{1}{784} \sum_{r=0}^{27} \sum_{c=0}^{27} \left(\varepsilon_{r,c} - \left[\varepsilon_{\theta(x_1,1,3)}\right]_{r,c} \right)^2 = 1.134913. \tag{12}$$

This is a single-image, single-timestep estimate of the coordinate-averaged objective, not the full training expectation. Numerical values are rounded to six decimal places; the saved computation uses unrounded tensors.

3.3 LDM: Two-Stage Latent Diffusion

Rombach et al. [5] (Section 3) first train an autoencoder, then train diffusion in its frozen latent space. For an image $x \in \mathbb{R}^{H \times W \times C}$, the encoder $\mathcal{E}$ produces a spatial latent in $\mathbb{R}^{h \times w \times c}$, where h, w, c are latent height, width, and channels. The downsampling factor is $f = \frac{H}{h} = \frac{W}{w}$. Reconstruction and generation are distinct:

$$z_0 = \mathcal{E}(x), \quad \tilde{x} = \mathcal{D}(\mathcal{E}(x)), \quad x_{\text{gen}} = \mathcal{D}(z_0^{\text{gen}}). \tag{13}$$

The generated latent z_0^{gen} comes from reverse diffusion, not from encoding a particular input image. Reducing each spatial dimension by f gives f^2 fewer spatial positions, while the scalar compression ratio also depends on channel count. For our $1 \times 28 \times 28$ images and $4 \times 7 \times 7$ latents, these reductions are $16\times$ and $4\times$, respectively.

The paper uses perceptual and adversarial reconstruction terms with either mild KL regularization or vector quantization. Our smaller VAE uses only pixel MSE and KL regularization, so it is an illustrative simplification. Its encoder outputs μ and $\log\sigma^2$ and samples $z = \mu + \sigma \odot \eta$, with independent $\eta \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$; $\odot$ denotes coordinatewise multiplication. The implemented objective is

$$L_{\text{VAE}} = \frac{1}{D}\|x - \tilde{x}\|^2 - \frac{\beta_{\text{KL}}}{2D_z}\sum_{j=1}^{D_z}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right), \tag{14}$$

where $D_z = h \times w \times c$, $\beta_{\text{KL}} = 5 \times 10^{-4}$, and both terms are also averaged over the batch. A mild KL penalty regularizes the latent distribution without guaranteeing exact standard normality [6]. For diffusion, the code caches deterministic encoder means and uses $z_0 = s\mu(x)$, with recorded global scale $s = 1.160420$. Generated latents are divided by s before decoding. The frozen VAE is not optimized with the denoiser.

Forward noising follows Eq. 3 with x replaced by z. The conditional latent objective is Eq. (3) of [5]:

$$L_{\text{LDM}} = \mathbb{E}_{x,y,t,\varepsilon}\left[\left\|\varepsilon - \varepsilon_{\theta\left(z_t, t, \tau_{\theta(y)}\right)}\right\|^2\right]. \tag{15}$$

Here (x, y) is a data-label pair, t is uniform, ε is independent standard Gaussian noise, and τ_θ encodes the condition. A smaller latent usually reduces denoiser arithmetic, but total cost also includes autoencoder training, encoding, and decoding. Compression and runtime depend on the setting.

Stage 1: Train the autoencoder

$$x \to \mathcal{E} \to (\mu, \sigma) \to z = \mu + \sigma \odot \eta \to \mathcal{D} \to \tilde{x}$$

Reconstruction loss + KL regularization; $\eta \sim \mathcal{N}(0, \boldsymbol{I})$

Stage 2: Freeze the autoencoder; train latent diffusion

$$x \to \mathcal{E} \to z_0 \to z_t \to \varepsilon_{\theta\left(z_t, t, \tau_{\theta(y)}\right)}$$

Add known Gaussian noise; optimize the noise-prediction error.

Generation: No input image is required

$$z_T \sim \mathcal{N}(0, \boldsymbol{I}) \quad \to \quad z_{T-1} \quad \to \quad \ldots \quad \to \quad z_0^{\text{gen}} \to \mathcal{D} \to x_{\text{gen}}$$

Repeat: predict noise, update latent, and add reverse noise if $t > 1$.

Condition $y \to \tau_{\theta(y)}$ supplies keys/values to the denoiser.

Figure 2: Two-stage pipeline drawn by the authors based on [5], Figure 3. Training uses encoded data; generation begins from random latent noise and ends with one decoder pass. The diagram shows the KL-regularized variant.

For token conditioning, let $F_i = \varphi_{i(z_t)} \in \mathbb{R}^{N \times d_i}$ be flattened U-Net features at layer i, with $N = h_i w_i$ spatial positions. The encoder output $U = \tau_{\theta(y)} \in \mathbb{R}^{M \times d_\tau}$ contains M tokens. For one attention head of width d, the row-oriented projections are

$$Q = F_i W_Q, \quad K = UW_K, \quad V = UW_V, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \tag{16}$$

The learned matrices have shapes $W_Q \in \mathbb{R}^{d_i \times d}$ and $W_K, W_V \in \mathbb{R}^{d_\tau \times d}$. Thus $Q \in \mathbb{R}^{N \times d}$, $K, V \in \mathbb{R}^{M \times d}$, and softmax normalizes each row over the M conditioning tokens. Multi-head attention repeats this

operation for separate heads. The original text-conditioned LDM uses a BERT tokenizer and transformer encoder; spatial conditions such as masks or low-resolution images can instead enter through concatenation [5].

Our two U-Nets both add the class embedding to the time embedding and apply four-head cross-attention after the first encoder block and at the bottleneck. Each class is represented by only one token ($M = 1$). Softmax over one key is always one, so the attention output is a class-dependent value broadcast across positions. This can supply class information, but does not demonstrate the token-selection or spatial alignment possible with a multi-token text encoder.

4 Results and Discussion

4.1 Team Illustrative Experiments

4.1.1 Experimental Setup

We analyze saved runs on the 60,000-image MNIST training split [9]. The run logs record PyTorch 2.11.0, CUDA 13.0, and one NVIDIA RTX PRO 6000 Blackwell Server Edition GPU. Both denoisers use the same compact U-Net design: base width 32, time-embedding dimension 64, two coordinate input channels, and additive class conditioning together with single-token cross-attention. They have different input/output channel counts and parameter totals. Each denoiser was trained for 50 epochs with batch size 64, seed 42, AdamW weight decay 10^{-4}, and cosine learning-rate decay to 10^{-5}. Initial learning rates differ: 0.002 for pixel diffusion and 0.0025 for latent diffusion. The VAE was trained for 30 epochs at learning rate 0.003. Appendix A records commands and measurement scope. A fixed seed aids repeatability but does not guarantee identical results across devices or software versions.

4.1.2 Forward Noising and Reverse Generation

The forward demonstration uses five mathematical timesteps: 50, 200, 400, 700, and 1000. A shared noise tensor is used in Eq. 3, giving valid marginal samples coupled for visualization rather than a sampled Markov trajectory. The signal/noise coefficients are approximately $\frac{0.9854}{0.1702}$ at step 50 and $\frac{0.4418}{0.8971}$ at step 400. At step 1000 the signal coefficient is approximately 0.00635, leaving a small residual signal. For display only, intensities are mapped to $[0, 1]$ and clipped; numerical diffusion states are not restricted to that interval.

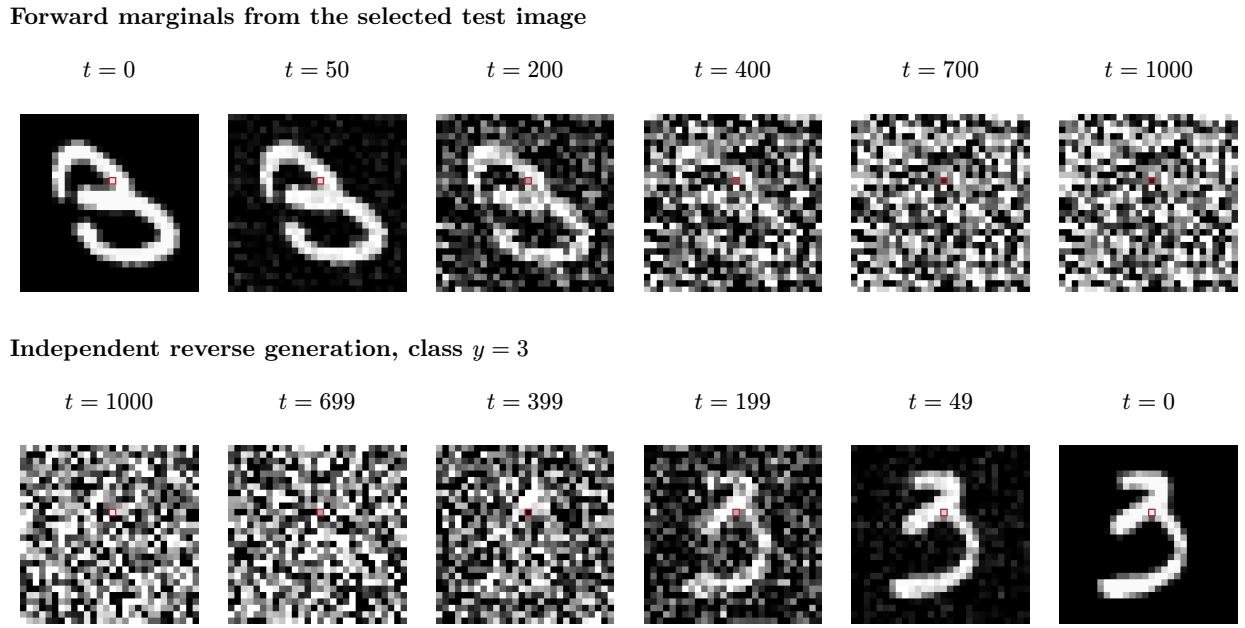


Figure 3: Team-generated MNIST demonstration [9]. Top: clean image and five forward noise levels. Bottom: an independent, class-3 ancestral generation, shown at states $1000, 699, 399, 199, 49, 0$. Red boxes mark zero-based coordinate $(10, 14)$; the rows do not show recovery of the same image. Source: `step2_train_ddpm.py`; snapshot labels corrected from the recorder's post-transition convention.

The reverse row in Figure 3 starts from independent Gaussian noise and uses 1,000 learned transitions. A recognizable digit emerges as sampling proceeds. The marked coordinate links the location to the worked example, not the identity of an original pixel. Intermediate states are labeled after their recorded transition: code index 699 produces mathematical state 699, whereas index 999 records the initial state before sampling.

4.2 Comparison of Pixel-Space and Latent Diffusion

4.2.1 Reconstruction and Class Conditioning

The VAE reduces each image to 196 latent coordinates. On the single selected test image, deterministic reconstruction gives MSE 9.7833×10^{-4} and Peak Signal-to-Noise Ratio (PSNR) 30.10 dB, computed on intensities in $[0, 1]$ as $10 \log_{10}\left(\frac{1}{\text{MSE}}\right)$. Figure 4 shows that this example retains the digit's visible structure. This is not a held-out-set average and does not establish that compression is harmless for all images.

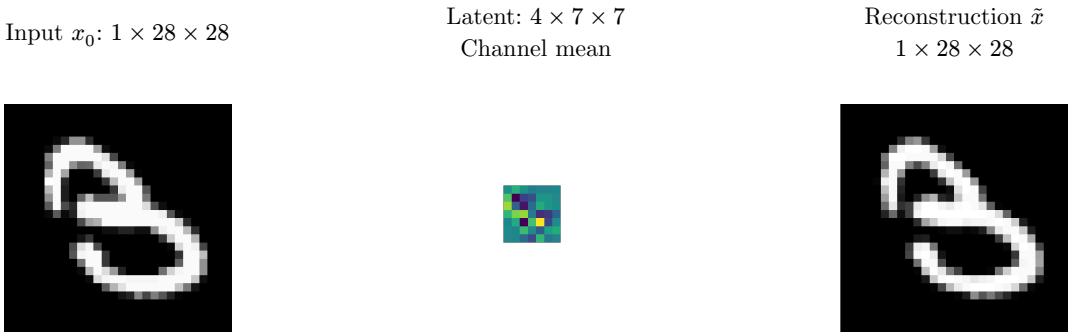


Figure 4: Team-generated VAE reconstruction example on MNIST [9], from `step3_train_ldm.py`. The center panel is the channel mean of a $4 \times 7 \times 7$ latent, min-max normalized for display. Its $\frac{1}{16}$ area illustrates spatial reduction; scalar storage decreases by $4\times$.

Class labels enter through additive embeddings and single-token cross-attention.

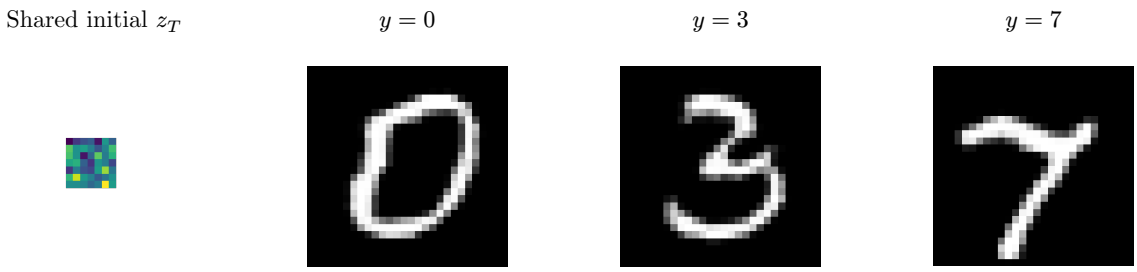


Figure 5: Team-generated class-conditioned samples from `step3_train_ldm.py` on MNIST [9]. Conditions are integer class labels, not free-form text. Initial latent noise is shared, but subsequent reverse-step noise differs across samples. Both additive conditioning and single-token cross-attention are active.

Figure 5 shows outputs for labels 0, 3, and 7. These recognizable examples illustrate class-conditioned generation, but cannot isolate the effect of attention from additive conditioning or subsequent stochastic draws. Because $M = 1$, no varying token-selection weights are learned in this demonstration. General text-prompt alignment and causal comparisons between conditioning mechanisms remain outside its scope.

4.2.2 Recorded Compute and Runtime

Table 3 separates measurements from the literature discussed in Section 2. The table reports the checked-in run statistics. Conditions are similar rather than identical, especially learning rates and cached latent preprocessing. Estimated arithmetic counts convolution and linear-layer multiply-accumulates (MACs), using two floating-point operations (FLOPs) per MAC and batch size one. Attention matrix products, softmax, normalization, activations, interpolation, sampler arithmetic, and VAE decoding are excluded.

Table 3: Team experiment records: `ddpm_pixel_stats.json`, `ldm_latent_stats.json`, and `vae_stats.json`. Values are single-run observations, not a matched quality or hardware benchmark. Times include only the workloads specified below.

Criterion	Pixel diffusion	Latent diffusion
Operating tensor	$1 \times 28 \times 28$	$4 \times 7 \times 7$
U-Net parameters	179,809	181,540
Additional VAE parameters	None	99,785
Counted FLOPs/U-Net evaluation	117.87 million	8.31 million
Counted FLOPs/1,000 evaluations	117.87 billion	8.31 billion
Mean denoiser epoch time	9.42 s	5.74 s
Denoiser epoch-time sum	471.2 s	286.9 s
VAE epoch-time sum	None	171.7 s
Combined epoch-time sum	471.2 s	458.6 s; preprocessing excluded
Recorded 1,000-step sampling time	1.592 s	1.715 s, including decode
Final training noise MSE	0.0210	0.1499; different space
Conditioning in both models	Additive class embedding and single-token cross-attention	

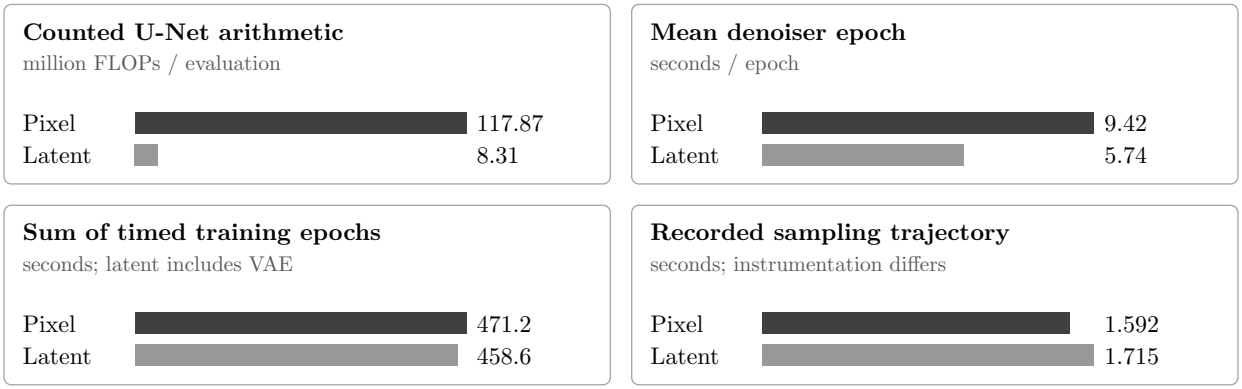


Figure 6: Authors' visualization of the same saved values as Table 3. The arithmetic estimate covers convolution/linear layers only. Training bars sum timed epochs and exclude data preparation and latent-cache construction. Sampling is one recorded trajectory per setting.

The latent U-Net has approximately $14.19\times$ fewer counted operations per step. Its recorded mean epoch time is 39.1% lower, equivalent to $1.64\times$ the epoch throughput. However, adding VAE epochs reduces the apparent advantage to $1.03\times$ for the sum of training-epoch times. Encoding the dataset and building the latent cache were not timed in this sum, so it is not an end-to-end comparison. The sampling observation is also mixed: latent generation is approximately 7.7% slower despite lower counted arithmetic. Small GPU workloads may be affected by launch overhead and memory traffic, and latent sampling includes decoding, but these explanations were not established by a kernel-level profile. The pixel timing also includes recording and transferring intermediate snapshots, unlike the latent timing; repeated measurements with matched instrumentation are needed for a controlled latency comparison.

The saved peak GPU allocations have been omitted from the comparison because the reset interval covers different stages, and evaluation-only runs can overwrite the memory field while retaining historical training times. The two noise losses also do not rank image quality: their input distributions, effective signal-to-noise ratios, and training configurations differ. Both targets are unit Gaussian noise, and black MNIST background becomes -1 after normalization. The loss gap does not measure entropy or prove the absence of underfitting.

4.3 Critical Analysis and Limitations

The observations support a reduction in counted denoiser arithmetic for this particular representation and backbone. They do not establish a universal speedup or transferable runtime ratio. Larger images also increase autoencoder costs. Reusing a pretrained encoder may amortize those costs across tasks, but

this was not measured here. Latent compression can discard detail and may constrain reconstruction; one favorable reconstruction cannot characterize that effect across the dataset [5].

Both implementations use 1,000-step ancestral sampling, with fresh reverse noise and no extra final-step noise. DDIM provides an alternative speed-quality trade-off in either representation [8]. Our study uses one seed, one dataset, small models, final-epoch checkpoints, and one recorded timing per configuration. No confidence intervals, distributional image-quality scores, class-accuracy statistics, or diversity evaluations were computed. The generated-to-reference PSNR values in the raw logs compare unrelated images and are not used as quality evidence. Published CIFAR-10 and ImageNet FIDs describe their original experiments, not these MNIST runs.

Training-data limitations matter beyond compute. MNIST contains handwritten digits rather than natural scenes, so its examples cannot establish performance on photographs, languages, or complex prompts [9]. Keras documents MNIST as copyrighted by Yann LeCun and Corinna Cortes and distributed under CC BY-SA 3.0 [10]; we cite the dataset and preserve attribution. This provenance does not imply that all generative-model training data has the same permissions.

The original LDM text-to-image experiment uses LAION-400M, while its other tasks use datasets such as ImageNet and Places [5]. Large image-text corpora may contain copyrighted or personal material and imbalanced representations. Later extraction research demonstrates that some diffusion systems can reproduce training images [11]; this is distinct from the original LDM experiments and was not tested here. Bias can also affect which appearances or associations a model generates. Dataset documentation, permission-aware curation, deduplication, and output audits can help address these concerns, but do not by themselves guarantee absence of memorization or bias. Our small visual demonstration does not measure either risk.

5 Conclusion

Learning to remove noise can support image generation because a denoiser learns statistical structure shared by training images at different noise levels. During sampling, this information guides a sequence of transitions from random noise toward the learned data distribution. DDPM demonstrates this approach in pixel space, while LDM applies it to a compressed representation and decodes the generated latent into an image. Conditioning provides additional information about the desired output, although our implementation tests class labels rather than text prompts.

The worked example and five noise levels illustrate how the signal decreases as Gaussian noise is added. The saved MNIST runs also show that a smaller latent representation can reduce counted U-Net arithmetic substantially. However, the recorded latent sampling time is slightly higher, and training requires an additional VAE stage. These observations show why representation size, actual latency, preprocessing, and reconstruction should be considered separately. They do not establish a quality ranking or predict full-scale performance. Within these limits, the report explains the main mechanisms of diffusion and the practical trade-offs involved in moving from pixels to latent space.

References

[1] I. J. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680. [Online]. Available: https://proceedings.neurips.cc/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html

[2] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved Techniques for Training GANs," *arXiv preprint arXiv:1606.03498*, 2016, [Online]. Available: https://arxiv.org/abs/1606.03498

[3] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *Proceedings of the 33rd International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 48. 2016, pp. 1747–1756. [Online]. Available: https://proceedings.mlr.press/v48/oord16.html

[4] J. Ho, A. N. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems*, 2020, pp. 6840–6851. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html

[5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10684–10695. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html

[6] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations*, 2014. [Online]. Available: https://arxiv.org/abs/1312.6114

[7] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *Proceedings of the 32nd International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 2256–2265. [Online]. Available: https://proceedings.mlr.press/v37/sohl-dickstein15.html

[8] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2010.02502

[9] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits." Accessed: Nov. 10, 2026. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[10] Keras, "MNIST Digits Classification Dataset." Accessed: Nov. 10, 2026. [Online]. Available: https://keras.io/api/datasets/mnist/

[11] N. Carlini *et al.*, "Extracting Training Data from Diffusion Models," *arXiv preprint arXiv:2301.13188*, 2023, [Online]. Available: https://arxiv.org/abs/2301.13188